



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

گزارش کار آزمایش چهارم

عنوان:

طراحی و پیاده‌سازی پشته

نام درس

آزمایشگاه طراحی سامانه‌های رقمی

تابستان ۱۴۰۵

استاد

دکتر اجلالی

دستیار استاد

علی اثنی‌عشری

گروه ۲

علی سلطانی - ۴۰۳۱۰۶۰۹۲

یگانه کربلایی - ۴۰۳۱۰۶۵۲۷

محمد مهدی مرادی - ۴۰۳۱۰۶۶۸۱

فهرست مطالب

۱	هدف آزمایش	۳
۲	شرح مسئله و تصمیم‌های طراحی	۳
۱.۲	رفتار پشته	۳
۲.۲	ورودی‌ها و خروجی‌ها	۴
۳.۲	نکته مربوط به سیگنال Empty	۴
۴.۲	رفتار هم‌زمان دو فرمان	۴
۳	طراحی پشته با عمق ۸ و پهنای ۴ بیت	۵
۱.۳	ساختار داخلی	۵
۲.۳	کد Verilog نسخه ثابت	۵
۴	سنتز و مشاهده ساختار نسخه ثابت در Quartus	۷
۱.۴	روند انجام کار در Quartus	۷
۲.۴	نمای RTL	۸
۳.۴	نمای پس از نگاشت	۹
۴.۴	نمای پس از جایابی و تطبیق	۱۰
۵.۴	نتیجه کامپایل نسخه ثابت	۱۰
۵	آزمون پشته ثابت	۱۱
۱.۵	ساختار فایل آزمون	۱۱
۲.۵	شکل موج فایل آزمون ثابت	۱۲
۶	توسعه پارامتری پشته	۱۲
۱.۶	پارامترهای قابل تنظیم	۱۲
۲.۶	کد Verilog نسخه پارامتری	۱۳
۷	سنتز نسخه پارامتری در Quartus	۱۵
۱.۷	پشته پارامتری با DEPTH=8 و WIDTH=4	۱۵
۲.۷	پشته پارامتری با DEPTH=4 و WIDTH=2	۱۷
۳.۷	نتیجه کامپایل نسخه پارامتری	۱۸
۸	فایل آزمون جامع نسخه پارامتری	۱۹
۱.۸	هدف از طراحی فایل آزمون جامع	۱۹
۲.۸	روند اجرای هر عملیات	۱۹
۳.۸	حالت‌های جهت‌دار و مرزی	۱۹
۴.۸	آزمون تنش تصادفی	۲۰
۹	نتایج شبیه‌سازی نسخه پارامتری	۲۱
۱.۹	نتیجه WIDTH=4, DEPTH=8	۲۲
۲.۹	نتیجه WIDTH=2, DEPTH=4	۲۲
۳.۹	نتیجه WIDTH=8, DEPTH=8	۲۳

۲۳	۴.۹ نتیجه WIDTH=5, DEPTH=5
۲۴	۱۰ فایل آزمون نمایشی و تحلیل شکل موج ها
۲۴	۱.۱۰ چرا یک فایل آزمون کوتاه تر ساخته شد؟
۲۵	۲.۱۰ شکل موج پشته پارامتری با عمق ۸ و پهنای ۴ بیت
۲۶	۳.۱۰ شکل موج پشته پارامتری با عمق ۴ و پهنای ۲ بیت
۲۶	۱۱ بحث و جمع بندی نتایج
۲۷	۱۲ فایل های تحویلی

۱ هدف آزمایش

هدف این آزمایش، آشنایی با نحوه پیاده‌سازی داده‌ساختار پشته^۱ در زبان Verilog و سپس توسعه همان طراحی به صورت پارامتری است. در مرحله نخست، پشته‌ای با عمق پشته^۲ برابر ۸ و پهنای داده^۳ برابر ۴ بیت طراحی می‌شود. در مرحله دوم، دو مشخصه DEPTH و WIDTH به پیمانه^۴ افزوده می‌شوند تا کاربر بتواند پیش از مرحله سنتز^۵ و تولید جریان داده^۶، اندازه پشته را تعیین کند.

هدف اصلی مرحله پارامتری این است که تغییر عمق یا پهنای داده، تنها با تغییر مقدار مشخصه‌ها انجام شود و هیچ نیازی به اصلاح دستی ساختار داخلی مدار وجود نداشته باشد. برای ارزیابی درستی طراحی نیز علاوه بر آزمون‌های جهت‌دار، مجموعه‌ای از حالت‌های مرزی^۷ و یک آزمون تنش تصادفی^۸ اجرا شده است.

۲ شرح مسئله و تصمیم‌های طراحی

۱.۲ رفتار پشته

پشته یک ساختار ذخیره‌سازی با ترتیب «آخرین ورودی، نخستین خروجی»^۹ است. در این آزمایش، داده‌ها با فرمان درج^{۱۰} در بالای پشته قرار می‌گیرند و با فرمان برداشت^{۱۱} از همان ناحیه خارج می‌شوند. برای نگهداری وضعیت پشته از یک اشاره‌گر^{۱۲} استفاده شده است که تعداد عناصر معتبر موجود در پشته را نشان می‌دهد.

در نسخه ثابت، مقدار اشاره‌گر از صفر تا ۸ تغییر می‌کند. بنابراین:

- اگر اشاره‌گر صفر باشد، پشته خالی است؛
- اگر اشاره‌گر ۸ باشد، پشته پر است؛
- در سایر مقادیر، تعداد عناصر موجود برابر مقدار اشاره‌گر است.

¹Stack

²Stack Depth

³Data Width

⁴Module

⁵Synthesis

⁶Bitstream

⁷Edge Cases

⁸Random Stress Test

⁹Last-In First-Out (LIFO)

¹⁰Push

¹¹Pop

¹²Pointer

۲.۲ ورودی‌ها و خروجی‌ها

در این گزارش، «ساعت»^{۱۳} و «بازنشانی»^{۱۴} به ترتیب برای سیگنال زمان‌بندی و بازگرداندن مدار به وضعیت آغازین به کار می‌روند. جدول ۱ رابط پیمانه را برای نسخه ثابت نشان می‌دهد.

نوع	نام سیگنال	شرح
ورودی	Clk	سیگنال ساعت؛ عملیات پشته در لبه بالارونده آن انجام می‌شود.
ورودی	RstN	سیگنال بازنشانی فعال‌پایین و ناهم‌زمان.
ورودی	Data_In[3:0]	داده چهار بیتی ورودی.
ورودی	push	فرمان درج داده در پشته.
ورودی	pop	فرمان برداشت داده از پشته.
خروجی	Data_Out[3:0]	آخرین داده برداشته‌شده از پشته.
خروجی	Full	در صورت پر بودن پشته برابر یک می‌شود.
خروجی	Empty	در صورت خالی بودن پشته برابر یک می‌شود.

جدول ۱: ورودی‌ها و خروجی‌های پشته با عمق ۸ و پهنای ۴ بیت

۳.۲ نکته مربوط به سیگنال Empty

در متن صورت آزمایش عبارت Empty=0 indicates that the stack is empty درج شده است. این عبارت با نام سیگنال Empty و تعریف رایج پرچم خالی‌بودن سازگار نیست. در این پیاده‌سازی، تعریف زیر به‌طور صریح انتخاب شده است:

$$\text{پشته خالی است.} \iff \text{Empty} = 1$$

تمام فایل‌های آزمون نیز بر اساس همین تعریف نوشته شده‌اند. این تصمیم در گزارش ذکر می‌شود تا رفتار مدار در این نقطه کاملاً مشخص باشد و ابهامی در تفسیر نتایج ایجاد نشود.

۴.۲ رفتار هم‌زمان دو فرمان

در صورت فعال‌شدن هم‌زمان push و pop، رفتار زیر انتخاب شده است:

۱. اگر پشته خالی باشد، امکان برداشت وجود ندارد؛ بنابراین فقط داده جدید درج می‌شود و تعداد عناصر از صفر به یک می‌رسد.

۲. اگر پشته خالی نباشد، عنصر بالایی در Data_Out قرار می‌گیرد و همان خانه با Data_In جایگزین می‌شود؛ بنابراین تعداد عناصر پشته ثابت می‌ماند.

¹³Clock

¹⁴Reset

این تفسیر باعث می‌شود حالت هم‌زمان در وضعیت‌های خالی، عادی، تک‌عضوی و پر، رفتار تعریف‌شده و قابل‌آزمون داشته باشد.

۳ طراحی پشته با عمق ۸ و پهنای ۴ بیت

۱.۳ ساختار داخلی

حافظه پشته به‌صورت آرایه‌ای شامل ۸ خانه تعریف شده است که هر خانه یک بردار ۴ بیتی است. متغیر `pointer` تعداد عناصر ذخیره‌شده را نگه می‌دارد. دو پرچم وضعیت به‌صورت ترکیبی از روی اشاره‌گر ساخته می‌شوند:

$$\text{Empty} = (\text{pointer} = 0), \quad \text{Full} = (\text{pointer} = 8).$$

در رخداد لبه بالارونده ساعت، یکی از حالت‌های زیر اجرا می‌شود:

- درج: اگر پشته پر نباشد، داده در خانه `stack[pointer]` نوشته و اشاره‌گر یک واحد زیاد می‌شود.
 - برداشت: اگر پشته خالی نباشد، داده خانه `stack[pointer-1]` به خروجی منتقل و اشاره‌گر یک واحد کم می‌شود.
 - درج و برداشت هم‌زمان: مطابق تصمیم بخش قبل عمل می‌شود.
 - حالت سکون: در صورت صفر بودن هر دو فرمان، وضعیت پشته بدون تغییر باقی می‌ماند.
- بازنشانی به‌صورت فعال‌پایین و ناهم‌زمان پیاده‌سازی شده است. بنابراین با صفرشدن `RstN`، بدون انتظار برای لبه ساعت، اشاره‌گر، خروجی و تمام خانه‌های حافظه صفر می‌شوند.

۲.۳ کد Verilog نسخه ثابت

کد کامل پیمانه اصلی در فایل `stack.v` قرار دارد:

```
1 module stack (  
2     input Clk,  
3     input RstN,  
4     input push,  
5     input pop,  
6     input [3:0] Data_In,  
7     output Full,  
8     output Empty,  
9     output reg [3:0] Data_Out  
10 );  
11  
12 // 8 entries, each 4 bits wide  
13 reg [3:0] stack [0:7];  
14  
15 // Number of stored elements (0 to 8)
```

```

16  reg [3:0] pointer;
17
18  integer i;
19
20  // Stack status flags
21  assign Empty = (pointer == 0);
22  assign Full  = (pointer == 8);
23
24  // Stack operations
25  always @(posedge Clk or negedge RstN) begin
26
27      // Active-low asynchronous reset
28      if (!RstN) begin
29          pointer <= 4'd0;
30          Data_Out <= 4'd0;
31
32          // Clear stack memory
33          for (i = 0; i < 8; i = i + 1) begin
34              stack[i] <= 4'd0;
35          end
36      end
37
38      else begin
39
40          // Simultaneous push and pop
41          if (push && pop) begin
42
43              // If empty, only push the new data
44              if (Empty) begin
45                  stack[pointer] <= Data_In;
46                  pointer <= pointer + 1;
47              end
48
49              // Replace the top element
50              else begin
51                  Data_Out <= stack[pointer - 1];
52                  stack[pointer - 1] <= Data_In;
53              end
54          end
55
56          // Push operation
57          else if (push) begin
58              if (!Full) begin
59                  stack[pointer] <= Data_In;
60                  pointer <= pointer + 1;
61              end
62          end

```

```

63
64 // Pop operation
65 else if (pop) begin
66     if (!Empty) begin
67         Data_Out <= stack[pointer - 1];
68         pointer <= pointer - 1;
69     end
70 end
71
72 end
73 end
74
75 endmodule

```

کد ۱: پیاده‌سازی پشته ثابت با عمق ۸ و پهنای ۴ بیت

۴ سنتز و مشاهده ساختار نسخه ثابت در Quartus

۱.۴ روند انجام کار در Quartus

برای سنتز نسخه ثابت، ابتدا از مسیر

File > New Project Wizard

یک پروژه در Quartus ساخته شد و فایل stack.v به آن افزوده شد. نام پیمانه stack به عنوان موجودیت سطح بالا^{۱۵} انتخاب شد. برای بررسی اولیه قابل سنتز بودن طراحی، مرحله تحلیل و سنتز از مسیر

Processing > Start > Start Analysis & Synthesis

اجرا شد. سپس برای تولید نتایج مرحله تطبیق و امکان مشاهده نمای پس از جایابی، کامپایل کامل از مسیر

Processing > Start Compilation

انجام گرفت.

پس از تکمیل کامپایل، سه نمای اصلی از مسیرهای زیر باز شدند:

Tools > Netlist Viewers > RTL Viewer

Tools > Netlist Viewers > Technology Map Viewer (Post-Mapping)

Tools > Netlist Viewers > Technology Map Viewer (Post-Fitting)

سه نمای بررسی شده عبارت‌اند از:

- نمای سطح انتقال ثبات^{۱۶} با گزینه RTL Viewer؛

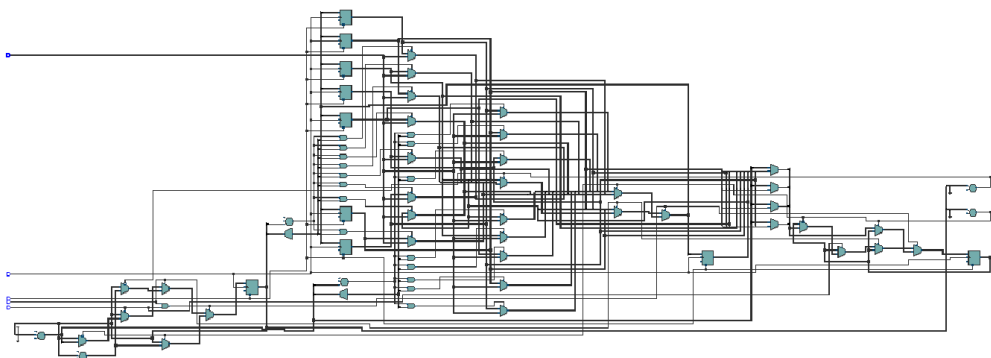
^{۱۵}Top-Level Entity

^{۱۶}Register-Transfer Level (RTL)

- نمای نگاشت فناوری^{۱۷} پس از نگاشت^{۱۸}؛
- نمای نگاشت فناوری پس از جایابی و تطبیق^{۱۹}.

نمای RTL ساختار منطقی حاصل از توصیف رفتاری را نشان می‌دهد و شامل ثبات‌ها، انتخاب‌گرها، مقایسه‌گرها و مسیرهای داده است. نمای پس از نگاشت نشان می‌دهد این منطق چگونه روی منابع منطقی فناوری هدف نگاشت شده است. در نمای پس از جایابی و تطبیق نیز ساختار پس از مرحله تطبیق نهایی با منابع FPGA مشاهده می‌شود.

۲.۴ نمای RTL



شکل ۱: نمای RTL پشته ثابت با عمق ۸ و پهنای ۴ بیت

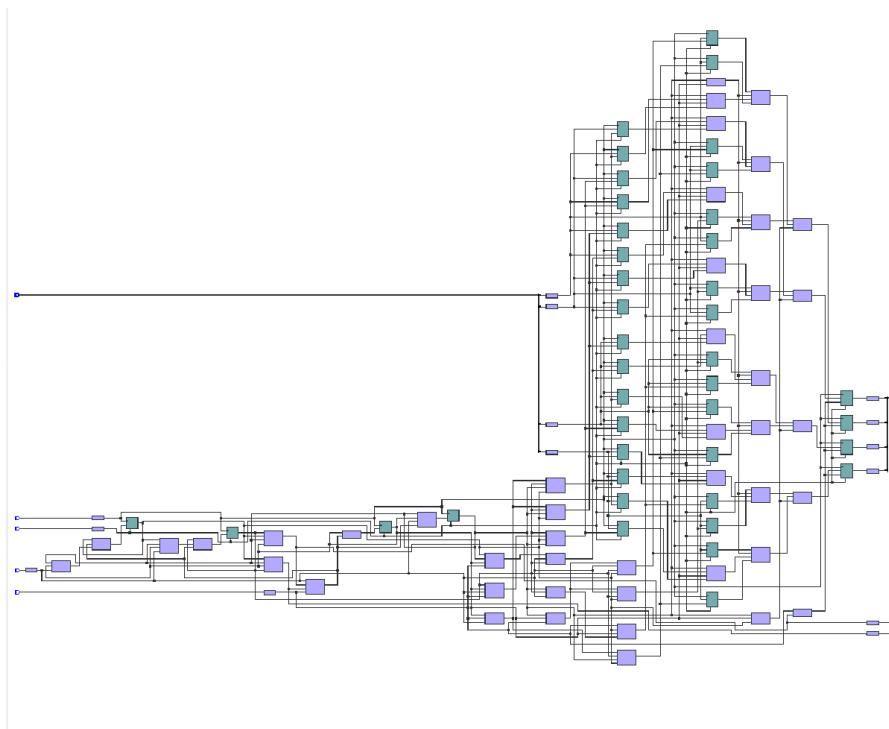
شکل ۱ نشان می‌دهد توصیف آرایه‌ای پشته پس از سنتز به مجموعه‌ای از ثبات‌ها، منطق انتخاب، مقایسه وضعیت اشاره‌گر و مسیر خروجی تبدیل شده است. شلوغی این نما طبیعی است، زیرا Quartus روابط رفتاری مانند انتخاب خانه بالای پشته و کنترل نوشتن روی خانه‌ها را به منطق صریح سخت‌افزاری تبدیل می‌کند.

¹⁷Technology Map Viewer

¹⁸Post-Mapping

¹⁹Post-Fitting

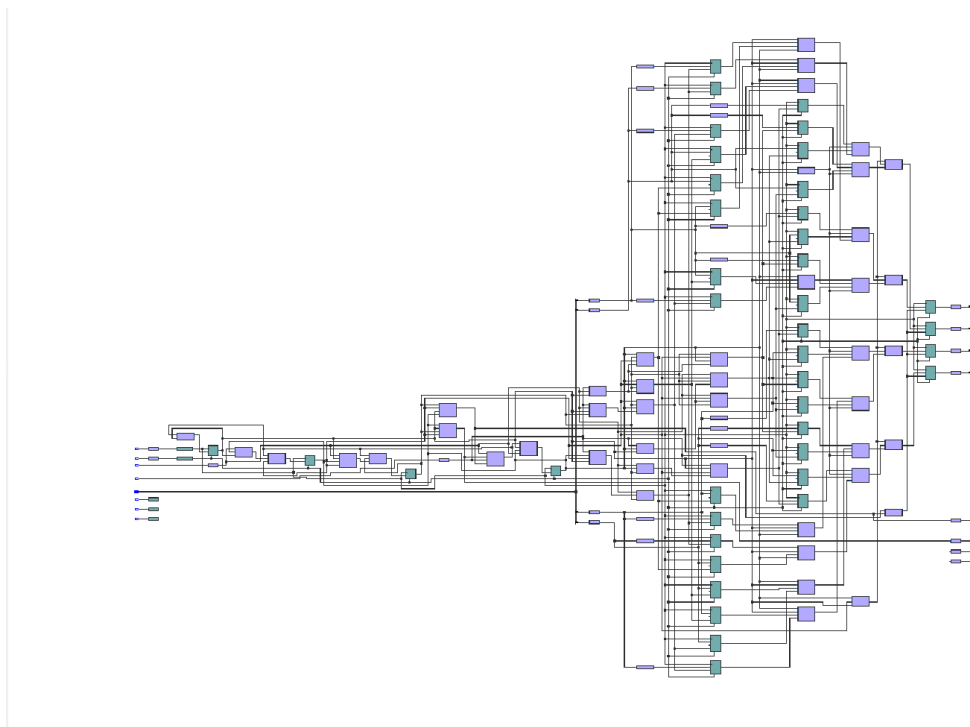
۳.۴ نمای پس از نگاشت



شکل ۲: نمای نگاشت فناوری پس از نگاشت برای پشته ثابت

در شکل ۲ منطق طراحی شده پس از مرحله نگاشت روی منابع منطقی هدف دیده می شود. در این مرحله بخشی از بهینه سازی های سنتز اعمال شده و ساختار نسبت به توصیف رفتاری به سطح فناوری نزدیک تر شده است.

۴.۴ نمای پس از جایابی و تطبیق



شکل ۳: نمای نگاشت فناوری پس از جایابی و تطبیق برای پشته ثابت

شکل ۳ نتیجه مرحله نهایی تر پیاده‌سازی را نشان می‌دهد. این نما برای اطمینان از آن مفید است که مدار تنها در سطح رفتاری صحیح نیست، بلکه تا مرحله تطبیق با منابع سخت‌افزاری نیز با موفقیت پیش رفته است.

۵.۴ نتیجه کامپایل نسخه ثابت

```

Analyzing Slow 1200mV BSC Model
332148 Timing requirements not met
332146 Worst-case setup slack is -1.651
332146 Worst-case hold slack is 0.356
332140 No Recovery paths to report
332140 No Removal paths to report
332146 Worst-case minimum pulse width slack is -3.000
Analyzing Slow 1200mV OC Model
334003 Started post-fitting delay annotation
334004 Delay annotation completed successfully
332123 Deriving Clock Uncertainty. Please refer to report_sdc in the Timing Analyzer to see clock uncertainties.
332148 Timing requirements not met
332146 Worst-case setup slack is -1.348
332146 Worst-case hold slack is 0.310
332140 No Recovery paths to report
332140 No Removal paths to report
332146 Worst-case minimum pulse width slack is -3.000
Analyzing Fast 1200mV OC Model
332123 Deriving Clock Uncertainty. Please refer to report_sdc in the Timing Analyzer to see clock uncertainties.
332148 Timing requirements not met
332146 Worst-case setup slack is -0.507
332146 Worst-case hold slack is 0.186
332140 No Recovery paths to report
332140 No Removal paths to report
332146 Worst-case minimum pulse width slack is -3.000
332102 Design is not fully constrained for setup requirements
332102 Design is not fully constrained for hold requirements
Quartus Prime Timing Analyzer was successful. 0 errors, 5 warnings
Running Quartus Prime EDA Netlist Writer
Command: quartus.eda --read-settings.files-off --write-settings.files-off stack -c stack
18236 Number of processors has not been specified which may cause overloading on shared machines. Set the global assignment NUM_PARALLEL_PROCESSORS in your QSF to an appropriate value for best performance.
204019 Generated file stack.vo in folder "/home/mahdi/Documents/DSOL/experiment 4/quartus/stack/simulation/questa/" for EDA simulation tool
Quartus Prime EDA Netlist Writer was successful. 0 errors, 1 warning
293000 Quartus Prime Full Compilation was successful. 0 errors, 18 warnings
    
```

شکل ۴: پایان موفق کامپایل کامل پشته ثابت در Quartus با صفر خطا

مطابق شکل ۴، کامپایل کامل با 0 errors به پایان رسیده است. هشدارهای زمان‌بندی موجود در پنجره ناشی از کامل نبودن قیود زمانی پروژه هستند. از آنجا که این آزمایش به صورت مجازی انجام شده و برنامه زمانی و پایه‌های یک برد فیزیکی برای

آن تعریف نشده‌اند، این هشدارها به معنای شکست سنتز یا خطای عملکردی در طراحی نیستند.

۵ آزمون پشته ثابت

۱.۵ ساختار فایل آزمون

برای نسخه ثابت، فایل `stack_tb.v` نوشته شد. در این فایل سه وظیفه کمکی^{۲۰} برای بازنشانی، درج یک داده و برداشت همراه با بررسی خروجی تعریف شده است. وظیفه چهارم نیز حالت درج و برداشت هم‌زمان را اعمال می‌کند. به این ترتیب، هر حالت آزمون^{۲۱} با هم‌زمانی مشخص نسبت به لبه‌های ساعت اجرا می‌شود و در صورت مغایرت خروجی، شمارنده خطا افزایش می‌یابد.

این فایل آزمون تنها مسیر عادی را بررسی نمی‌کند و ۱۲ حالت آزمون مجزا برای پوشش حالت‌های مرزی دارد. جدول ۲ هدف هر آزمون را نشان می‌دهد.

جدول ۲: حالت‌های آزمون پشته ثابت

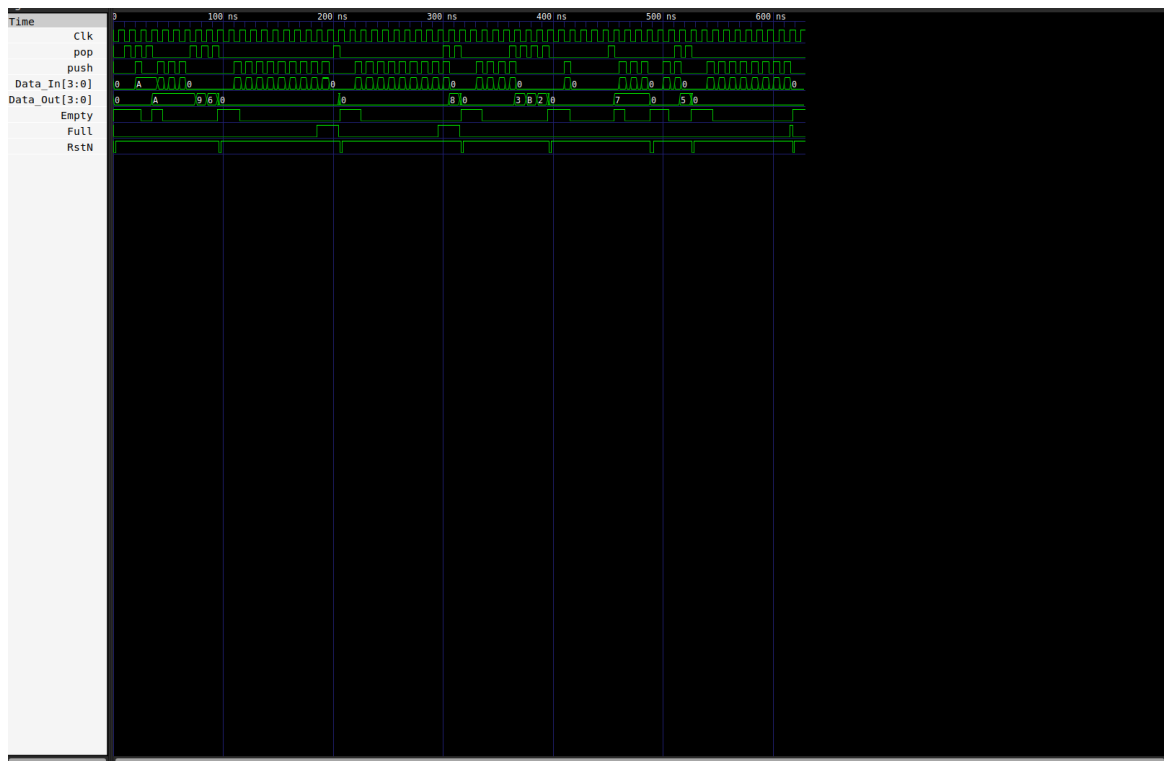
شماره	آزمون	هدف و حالت مرزی مورد بررسی
۱	بازنشانی	بررسی صفرشدن اشاره‌گر و خروجی و فعال‌شدن پرچم خالی‌بودن.
۲	برداشت از پشته خالی	بررسی جلوگیری از کم‌شدن اشاره‌گر در مرز صفر و حفظ وضعیت پشته.
۳	درج و برداشت هم‌زمان روی پشته خالی	بررسی تصمیم طراحی که در این حالت فقط درج انجام شود.
۴	رفتار LIFO عادی	درج چند داده و برداشت آن‌ها به ترتیب معکوس ورود.
۵	پرکردن پشته	بررسی تغییر اشاره‌گر تا مقدار ۸ و فعال‌شدن دقیق Full.
۶	درج روی پشته پر	بررسی جلوگیری از سرریز ^{۲۲} و اطمینان از نوشته‌نشدن داده اضافی.
۷	درج و برداشت هم‌زمان در حالت پر	بررسی جایگزینی عنصر بالایی بدون تغییر تعداد عناصر و باقی‌ماندن Full=1.
۸	درج و برداشت هم‌زمان در حالت عادی	بررسی خروج عنصر بالایی و جایگزینی آن با داده جدید بدون تخریب عناصر پایین‌تر.
۹	حالت سکون	بررسی ثابت‌ماندن داده‌ها در چند دوره ساعت هنگامی که هیچ فرمانی فعال نیست.
۱۰	بازنشانی در حالت غیرخالی	بررسی ماهیت ناهم‌زمان بازنشانی و پاک‌شدن فوری وضعیت.
۱۱	درج و برداشت هم‌زمان با یک عنصر	بررسی مرز مهم <code>pointer=1</code> و دسترسی صحیح به <code>pointer-1</code> .
۱۲	بازنشانی در حالت پر	بررسی انتقال مستقیم از مرز پر به وضعیت خالی و صفرشدن خروجی.

اجرای این فایل آزمون با Icarus Verilog انجام شد و نتیجه نهایی ALL TESTS PASSED به دست آمد.

²⁰Task

²¹Scenario

۲.۵ شکل موج فایل آزمون ثابت



شکل ۵: شکل موج اجرای فایل آزمون پشته ثابت در GTKWave

شکل ۵ مجموعه‌ای از حالت‌های آزمون در فایل آزمون ۱۲ حالت را نشان می‌دهد. در بخش‌هایی که چند پالس پیایی push مشاهده می‌شود، پشته تا مرز پر شدن پیش می‌رود و Full فعال می‌شود. سپس در بخش‌های برداشت، Data_Out داده‌های ذخیره‌شده را با ترتیب LIFO نمایش می‌دهد. پالس‌های کوتاه RstN نیز آزمون بازنشانی ناهم‌زمان در وضعیت‌های غیرخالی و پر هستند. هم‌زمانی push و pop در چند نقطه به‌طور عمدی ایجاد شده تا حالت‌های مرزی تعریف‌شده برای این دو فرمان بررسی شوند.

۶ توسعه پارامتری پشته

۱.۶ پارامترهای قابل تنظیم

برای حذف وابستگی طراحی به اعداد ثابت ۴ و ۸، دو مشخصه زیر تعریف شدند:

• WIDTH: پهنای هر داده ذخیره‌شده؛

• DEPTH: تعداد خانه‌های پشته.

در نتیجه آرایه حافظه به‌صورت زیر تعریف می‌شود:

```
reg [WIDTH-1:0] stack [0:DEPTH-1];
```

همچنین پهنای اشاره‌گر دیگر نباید از WIDTH گرفته شود، زیرا عرض داده و تعداد خانه‌های پشته دو مفهوم مستقل هستند. اشاره‌گر باید توانایی نمایش مقادیر صفر تا DEPTH را داشته باشد؛ بنابراین از مشخصه محلی^{۲۳} زیر استفاده شده است:

$$PTR_{WIDTH} = \lceil \log_2(DEPTH + 1) \rceil .$$

در Verilog این رابطه با $\$clog2(DEPTH + 1)$ محاسبه می‌شود. این انتخاب به‌ویژه برای عمق‌های غیرتوان دو مهم است. برای مثال، اگر $DEPTH=5$ باشد، اشاره‌گر باید مقادیر صفر تا پنج را نمایش دهد و به سه بیت نیاز دارد.

۲.۶ کد Verilog نسخه پارامتری

کد کامل پیمانه پارامتری در فایل stack_parametric.v آمده است:

```

1 module stack_parametric #(
2     parameter WIDTH = 2,
3     parameter DEPTH = 4
4 )
5 (
6     input Clk,
7     input RstN,
8     input push,
9     input pop,
10    input [WIDTH - 1:0] Data_In,
11    output Full,
12    output Empty,
13    output reg [WIDTH - 1:0] Data_Out
14 );
15
16 // DEPTH entries, each WIDTH bits wide
17 reg [WIDTH - 1:0] stack [0:DEPTH - 1];
18
19 // Number of stored elements (0 to DEPTH)
20 localparam PTR_WIDTH = $clog2(DEPTH + 1);
21 reg [PTR_WIDTH - 1:0] pointer;
22 integer i;
23
24 // Stack status flags
25 assign Empty = (pointer == 0);
26 assign Full = (pointer == DEPTH);
27
28 // Stack operations
29 always @(posedge Clk or negedge RstN) begin
30

```

²³Local Parameter

```

31      // Active-low asynchronous reset
32      if (!RstN) begin
33          pointer <= 0;
34          Data_Out <= 0;
35
36          // Clear stack memory
37          for (i = 0; i < DEPTH; i = i + 1) begin
38              stack[i] <= 0;
39          end
40      end
41
42      else begin
43
44          // Simultaneous push and pop
45          if (push && pop) begin
46
47              // If empty, only push the new data
48              if (Empty) begin
49                  stack[pointer] <= Data_In;
50                  pointer <= pointer + 1;
51              end
52
53              // Replace the top element
54              else begin
55                  Data_Out <= stack[pointer - 1];
56                  stack[pointer - 1] <= Data_In;
57              end
58          end
59
60          // Push operation
61          else if (push) begin
62              if (!Full) begin
63                  stack[pointer] <= Data_In;
64                  pointer <= pointer + 1;
65              end
66          end
67
68          // Pop operation
69          else if (pop) begin
70              if (!Empty) begin
71                  Data_Out <= stack[pointer - 1];
72                  pointer <= pointer - 1;
73              end
74          end
75
76      end
77  end

```

کد ۲: پیاده‌سازی پشته پارامتری

منطق عملیاتی این نسخه با نسخه ثابت یکسان است؛ تفاوت اصلی در ابعاد حافظه، ابعاد پورت‌های داده، حد پرشدن و عرض اشاره‌گر است. بنابراین تغییر مشخصه‌ها پیش از سنتز، سخت‌افزار جدید را بدون تغییر در الگوریتم داخلی تولید می‌کند.

۷ سنتز نسخه پارامتری در Quartus

برای نسخه پارامتری نیز یک پروژه Quartus با موجودیت سطح بالای `stack_parametric` ساخته شد و کامپایل کامل اجرا گردید. مسیر مشاهده نماها همان مسیر نسخه ثابت است:

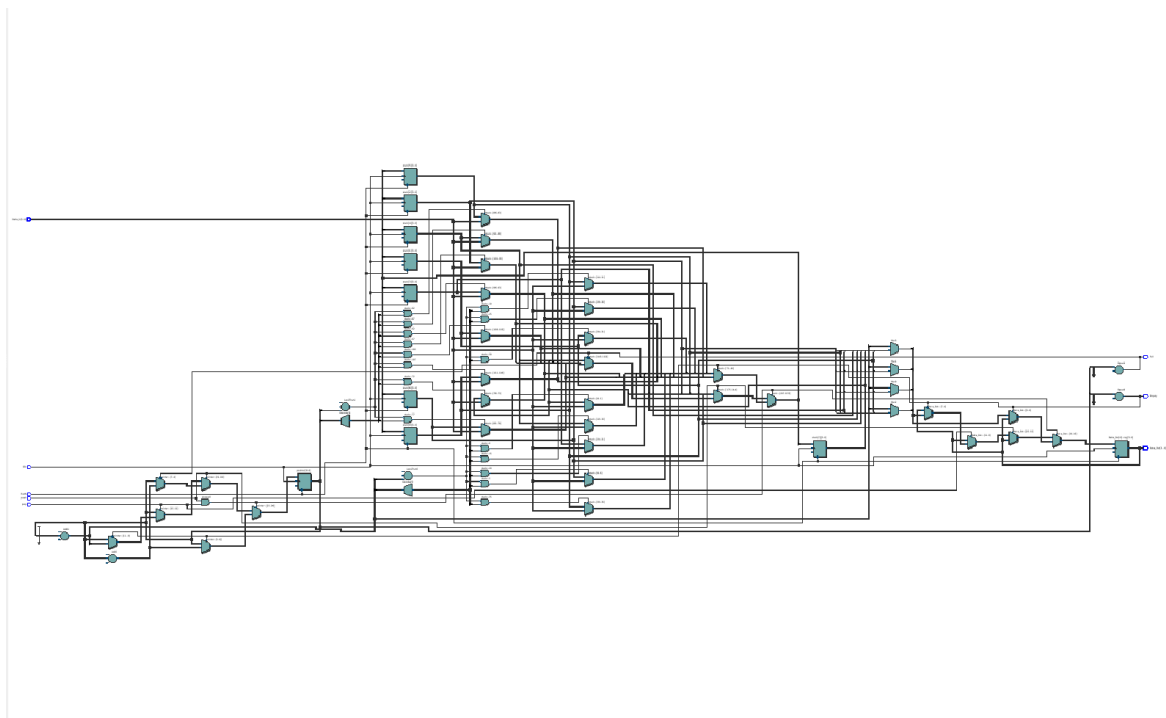
Tools > Netlist Viewers > RTL Viewer

و برای دو نمای فناوری:

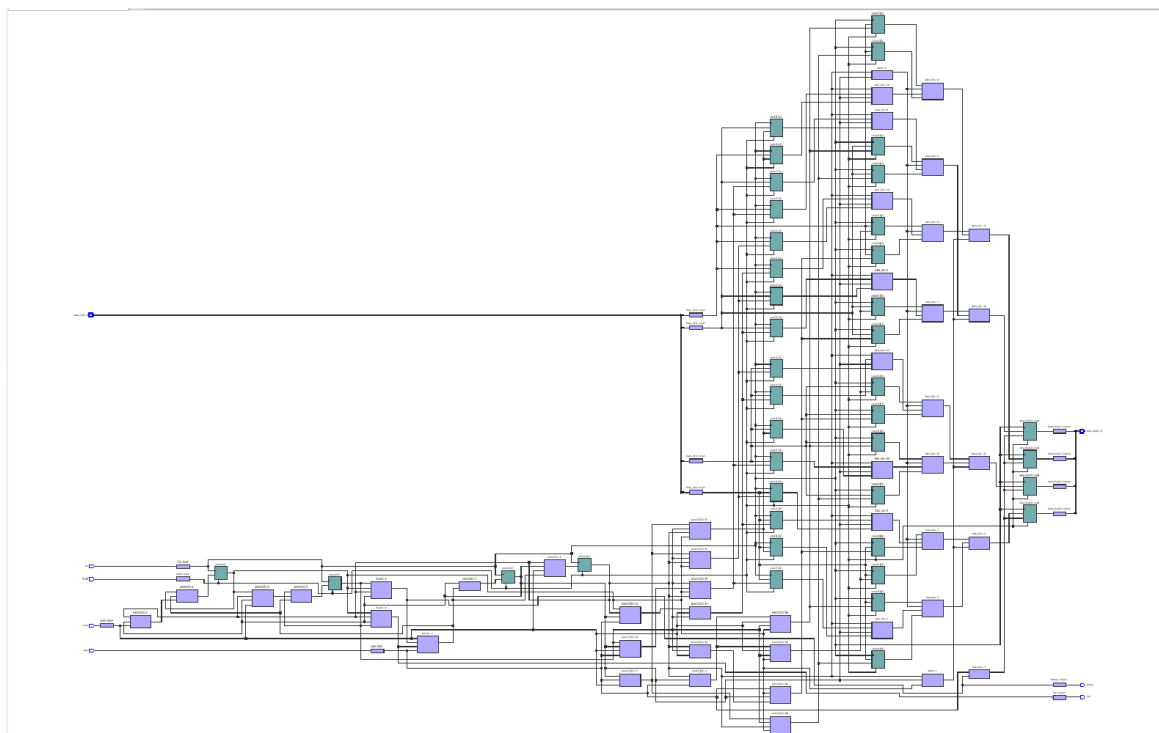
Tools > Netlist Viewers > Technology Map Viewer

۱.۷ پشته پارامتری با DEPTH=8 و WIDTH=4

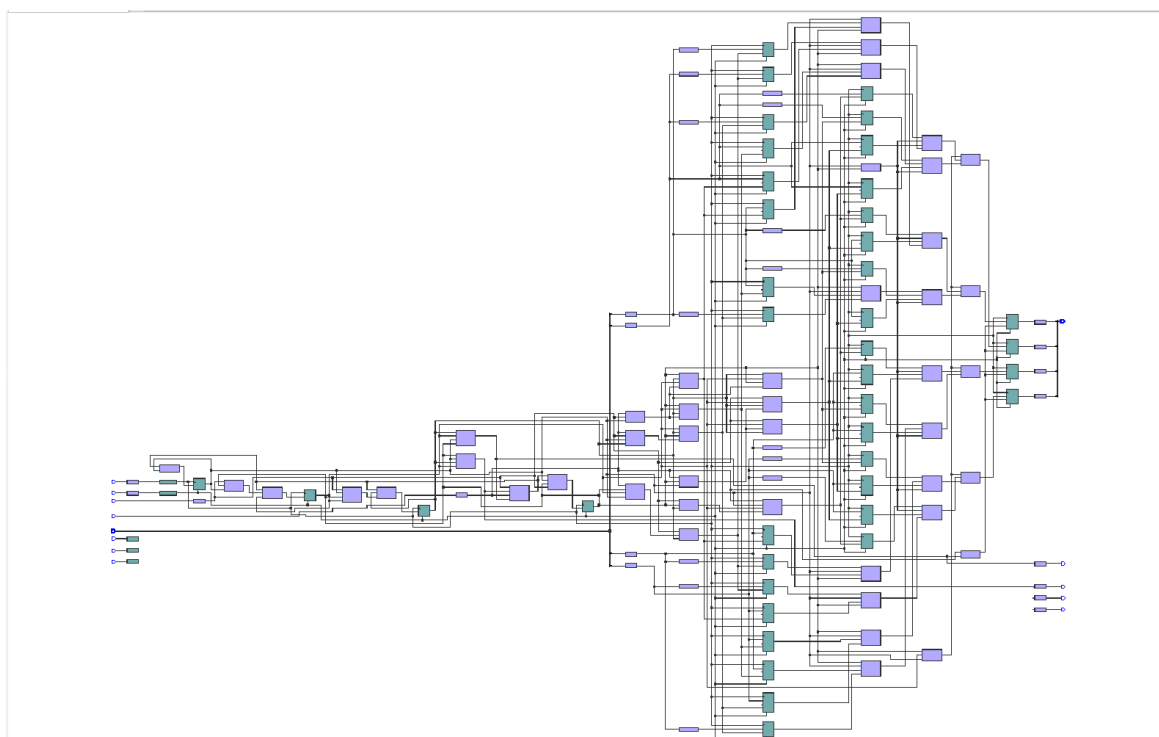
ابتدا مشخصه‌ها روی مقادیر پیش فرض صورت آزمایش قرار گرفتند. هدف این مرحله آن بود که نشان داده شود پارامتری‌شدن کد، رفتار و ظرفیت حالت ۸ خانه و داده ۴ بیتی را حفظ می‌کند.



شکل ۶: نمای RTL پشته پارامتری با عمق ۸ و پهنای ۴ بیت



شکل ۷: نمای پس از نگاشت پشته پارامتری با عمق ۸ و پهنای ۴ بیت

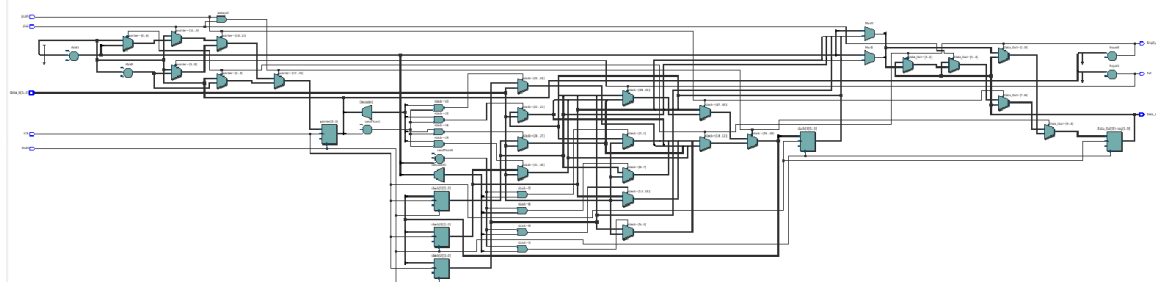


شکل ۸: نمای پس از جایابی و تطبیق پشته پارامتری با عمق ۸ و پهنای ۴ بیت

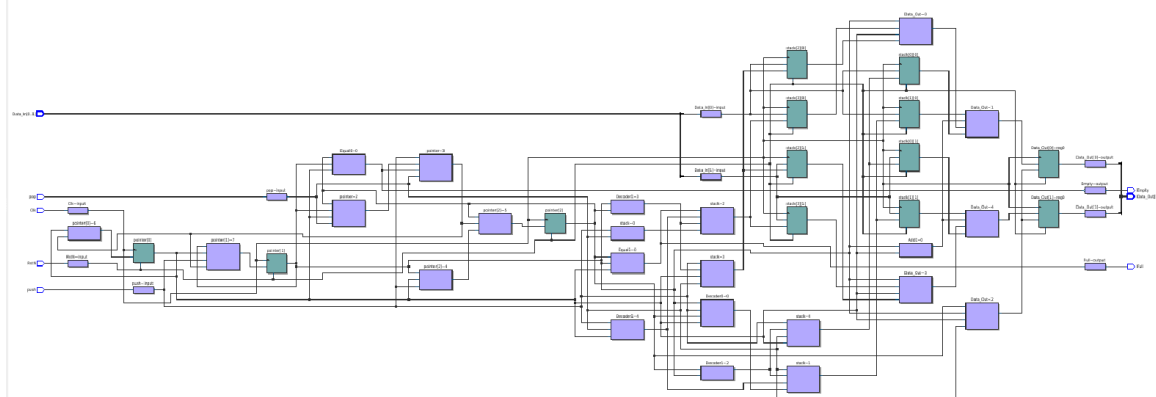
مقایسه شکل‌های ۶ تا ۸ با نماهای نسخه ثابت نشان می‌دهد که برای همان عمق و پهنای، سخت‌افزار حاصل از نسخه پارامتری از نظر مقیاس کلی با طراحی ثابت هم‌خوان است؛ در عین حال، ساختار کد دیگر به اعداد ثابت وابسته نیست.

۲.۷ پشته پارامتری با $DEPTH=4$ و $WIDTH=2$

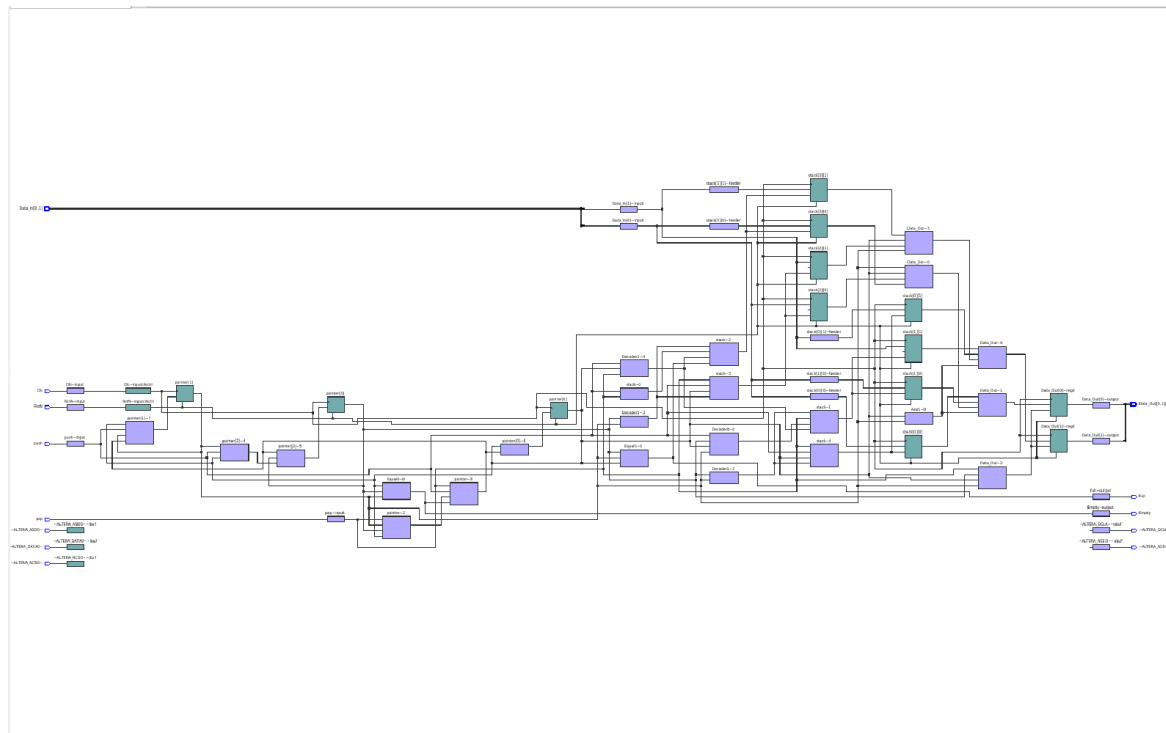
برای اثبات بازتولید خودکار سخت‌افزار، مقادیر مشخصه‌ها به عمق ۴ و پهنای ۲ بیت تغییر داده شد و پروژه دوباره سنتز شد.



شکل ۹: نمای RTL پشته پارامتری با عمق ۴ و پهنای ۲ بیت



شکل ۱۰: نمای پس از نگاشت پشته پارامتری با عمق ۴ و پهنای ۲ بیت



شکل ۱۱: نمای پس از جایابی و تطبیق پشته پارامتری با عمق ۴ و پهنای ۲ بیت

در این مجموعه، کاهش تعداد خانه‌ها از ۸ به ۴ و کاهش پهنای هر داده از ۴ به ۲ بیت باعث کوچک‌تر شدن شبکه منطقی و تعداد عناصر ذخیره‌سازی می‌شود. این تغییر در نماهای RTL و نگاشت فناوری قابل مشاهده است و خواسته اصلی صورت آزمایش، یعنی بازتولید خودکار سخت‌افزار با اندازه جدید، را نشان می‌دهد.

۳.۷ نتیجه کامپایل نسخه پارامتری

```

Analyzing Slow 1200mV BSC Model
332148 Timing requirements not met
332146 Worst-case setup slack is -1.651
332146 Worst-case hold slack is 0.356
332148 No Recovery paths to report
332148 No Removal paths to report
332146 Worst-case minimum pulse width slack is -3.000
Analyzing Slow 1200mV BC Model
334083 Started post-fitting delay annotation
334084 Delay annotation completed successfully
332123 Deriving Clock Uncertainty. Please refer to report_sdc in the Timing Analyzer to see clock uncertainties.
332148 Timing requirements not met
332146 Worst-case setup slack is -1.348
332146 Worst-case hold slack is 0.310
332148 No Recovery paths to report
332148 No Removal paths to report
332146 Worst-case minimum pulse width slack is -3.000
Analyzing Fast 1200mV BC Model
332123 Deriving Clock Uncertainty. Please refer to report_sdc in the Timing Analyzer to see clock uncertainties.
332148 Timing requirements not met
332146 Worst-case setup slack is -0.507
332146 Worst-case hold slack is 0.186
332148 No Recovery paths to report
332148 No Removal paths to report
332146 Worst-case minimum pulse width slack is -3.000
332182 Design is not fully constrained for setup requirements
332182 Design is not fully constrained for hold requirements
Quartus Prime Timing Analyzer was successful. 0 errors, 5 warnings
Running Quartus Prime EDA Netlist Writer
Command: quartus.eda --read settings.files=off --write settings.files=off stack_parametric -c stack_parametric
18236 Number of processors has not been specified which may cause overloading on shared machines. Set the global assignment NUM_PARALLEL_PROCESSORS in your OSF to an appropriate value for best performance.
264019 Generated file stack_parametric.vo in folder ~/home/mahdi/Documents/OSDL/experiment 4/quartus/stack_parametric/simulation/questa/ for EDA simulation tool
Quartus Prime EDA Netlist Writer was successful. 0 errors, 1 warning
293080 Quartus Prime Full Compilation was successful. 0 errors, 18 warnings

```

شکل ۱۲: پایان موفق کامپایل کامل نسخه پارامتری در Quartus با صفر خطا

شکل ۱۲ نشان می‌دهد نسخه پارامتری نیز با 0 errors کامپایل شده است. هشدارهای زمان‌بندی، همانند نسخه ثابت، ناشی از نبود قیود کامل زمانی در پروژه مجازی هستند و مانع سنتز طراحی نشده‌اند.

۸ فایل آزمون جامع نسخه پارامتری

۱.۸ هدف از طراحی فایل آزمون جامع

نسخه پارامتری باید برای چندین ترکیب از عمق و پهنا داده، بدون تغییر منطق آزمون قابل بررسی باشد. به همین دلیل فایل `stack_parametric_tb.v` نیز پارامتری نوشته شد و پیمانه تحت آزمون^{۲۴} با همان مقادیر `DEPTH` و `WIDTH` نمونه سازی می شود.

تفاوت مهم این فایل با یک آزمون ساده در آن است که فقط چند مقدار از پیش تعیین شده را با خروجی مقایسه نمی کند. در داخل فایل آزمون یک مدل مرجع^{۲۵} مستقل نگهداری می شود:

- آرایه `model_stack` وضعیت مورد انتظار پشته را ذخیره می کند؛
 - متغیر `model_count` تعداد عناصر مورد انتظار را نگه می دارد؛
 - بعد از هر عملیات، پرچم های `Full` و `Empty` با وضعیت مدل مرجع مقایسه می شوند؛
 - در برداشت و در حالت هم زمان، `Data_Out` با عنصر مورد انتظار مدل مرجع مقایسه می شود.
- به این ترتیب، آزمون از روی رفتار درست پشته نتیجه مورد انتظار را تولید می کند و لازم نیست برای هر عمق و پهنا، پاسخ ها به صورت دستی در کد نوشته شوند.

۲.۸ روند اجرای هر عملیات

وظیفه اصلی فایل آزمون با نام `apply_operation` سه ورودی دارد: فعال بودن درج، فعال بودن برداشت و مقدار داده ورودی. روند آن چنین است:

۱. در لبه پایین رونده ساعت، ورودی ها تنظیم می شوند تا پیش از لبه فعال مدار پایدار باشند؛
 ۲. در لبه بالارونده بعدی، پیمانه اصلی عملیات را انجام می دهد؛
 ۳. پس از یک تأخیر کوتاه، خروجی واقعی با مدل مرجع مقایسه می شود؛
 ۴. مدل مرجع مطابق همان عملیات به روزرسانی می شود؛
 ۵. در پایان، پرچم های پر و خالی بررسی و ورودی های کنترلی پاک می شوند.
- این روش از خطاهای ناشی از هم زمانی نامناسب فایل آزمون و مدار جلوگیری می کند و باعث می شود هر بررسی پس از تثبیت خروجی انجام شود.

۳.۸ حالت های جهت دار و مرزی

پیش از آزمون تصادفی، ۱۵ آزمون جهت دار اجرا می شوند تا حالت های مرزی مهم به طور قطعی پوشش داده شوند. آزمون شانزدهم نیز آزمون تنش تصادفی است.

²⁴Device Under Test (DUT)

²⁵Reference Model

جدول ۳: آزمون‌های جامع نسخه پارامتری

شماره	آزمون	هدف
۱	بازنشانی	بررسی وضعیت صفر، $Full=0$ ، $Empty=1$ و صفرشدن خروجی.
۲	برداشت در حالت خالی	جلوگیری از کم شدن تعداد عناصر از صفر و حفظ خروجی.
۳	درج نخستین عنصر	بررسی عبور از مرز خالی به غیر خالی و سپس صحت برداشت همان عنصر.
۴	درج و برداشت هم‌زمان در حالت خالی	بررسی اجرای فقط درج مطابق تصمیم طراحی.
۵	LIFO عادی	بررسی ترتیب صحیح داده‌ها در چند درج و برداشت.
۶	مرز پرشدن	بررسی اینکه $Full$ پیش از درج آخر فعال نشود و دقیقاً پس از رسیدن شمارنده به $DEPTH$ فعال شود.
۷	درج در حالت پر	بررسی جلوگیری از سرریز و سپس برداشت تمام داده‌ها برای اطمینان از تخریب نشدن محتوا.
۸	برداشت از حالت پر	بررسی اینکه با یک برداشت، $Full$ فوراً غیر فعال شود.
۹	درج و برداشت هم‌زمان در حالت پر	بررسی جایگزینی رأس و ثابت ماندن تعداد عناصر روی $DEPTH$.
۱۰	درج و برداشت هم‌زمان با یک عنصر	بررسی مرز $pointer=1$ و جایگزینی صحیح تنها عنصر.
۱۱	درج و برداشت هم‌زمان در حالت عادی	بررسی جایگزینی رأس بدون تغییر عناصر قبلی.
۱۲	مرز خالی شدن	بررسی فعال شدن دقیق $Empty$ پس از آخرین برداشت و سپس حفاظت در برابر برداشت اضافی.
۱۳	حالت سکون	بررسی ثابت ماندن وضعیت و خروجی در چند دوره ساعت بدون فرمان.
۱۴	بازنشانی در حالت غیر خالی	بررسی عملکرد ناهم‌زمان بازنشانی در میانه کار.
۱۵	بازنشانی در حالت پر	بررسی پاک شدن مستقیم وضعیت مرزی پر.
۱۶	آزمون تنش تصادفی	اجرای ۲۰۰ عملیات تصادفی از میان سکون، درج، برداشت و حالت هم‌زمان و مقایسه نتیجه پس از تک تک عملیات‌ها با مدل مرجع.

وجود آزمون‌های ۶، ۷، ۸ و ۱۲ برای کنترل دو مرز ۰ و $DEPTH$ اهمیت ویژه دارد. همچنین آزمون‌های ۴، ۹، ۱۰ و ۱۱ چهار وضعیت متفاوت فرمان‌های هم‌زمان را پوشش می‌دهند؛ بنابراین رفتار هم‌زمان فقط در یک حالت عادی آزموده نشده است.

۴.۸ آزمون تنش تصادفی

پس از آزمون‌های جهت دار، ۲۰۰ عملیات تصادفی اجرا می‌شود. در هر تکرار، یکی از چهار حالت زیر انتخاب می‌شود:

کد عملیات	عملیات
۰	سکون
۱	درج
۲	برداشت
۳	درج و برداشت همزمان

داده ورودی نیز تصادفی است. از آنجا که مدل مرجع بعد از هر عملیات به روزرسانی و مقایسه می شود، آزمون تصادفی تنها یک «اجرای بدون خطا» نیست؛ بلکه هر مرحله دارای بررسی خودکار است. در پایان نیز تمام عناصر باقی مانده برداشت می شوند و یک برداشت اضافی روی پشته خالی انجام می شود تا وضعیت نهایی نیز کنترل شود.

۹ نتایج شبیه سازی نسخه پارامتری

برای اثبات عمومی بودن طراحی، فایل آزمون جامع روی چهار ترکیب متفاوت اجرا شد. دو ترکیب نخست به ترتیب حالت اصلی صورت آزمایش و یک نسخه کوچک تر هستند. ترکیب $WIDTH=8$, $DEPTH=8$ تغییر پهنا را به تنهایی بررسی می کند و ترکیب $WIDTH=5$, $DEPTH=5$ نیز اهمیت ویژه دارد، زیرا عمق ۵ توان دو نیست.

پهنای داده	عمق پشته	تعداد بررسی ها	نتیجه
۴	۸	۱۲۴۴	ALL TESTS PASSED ✓
۲	۴	۱۱۳۲	ALL TESTS PASSED ✓
۸	۸	۱۲۴۴	ALL TESTS PASSED ✓
۵	۵	۱۱۶۰	ALL TESTS PASSED ✓

جدول ۴: خلاصه نتایج فایل آزمون جامع برای چهار پیکربندی

۱.۹ نتیجه WIDTH=4, DEPTH=8

```
● mahdi@mahdi-ASUS-Zenbook-14:~/Documents/DSDL/experiment 4/quartus/stack_parametric$ vvp sim.out
VCD info: dumpfile stack_tb.vcd opened for output.
=====
STACK TEST: WIDTH=4 DEPTH=8
=====
Test 1: Reset
Test 2: Pop when empty
Test 3: Push first element
Test 4: Push-Pop when empty
Test 5: Normal LIFO
Test 6: Full boundary
Test 7: Push when full
Test 8: Pop from full
Test 9: Push-Pop when full
Test 10: Push-Pop with one element
Test 11: Normal Push-Pop
Test 12: Empty boundary
Test 13: Idle state
Test 14: Reset when non-empty
Test 15: Reset when full
Test 16: Random stress test
=====
Configuration: WIDTH=4 DEPTH=8
Checks performed: 1244
ALL TESTS PASSED
=====
stack_parametric_tb.v:834: $finish called at 3156000 (1ps)
```

شکل ۱۳: عبور موفق ۱۲۴۴ بررسی برای پشته پارامتری با عمق ۸ و پهنای ۴ بیت

۲.۹ نتیجه WIDTH=2, DEPTH=4

```
● mahdi@mahdi-ASUS-Zenbook-14:~/Documents/DSDL/experiment 4/quartus/stack_parametric$ vvp sim.out
VCD info: dumpfile stack_tb.vcd opened for output.
=====
STACK TEST: WIDTH=2 DEPTH=4
=====
Test 1: Reset
Test 2: Pop when empty
Test 3: Push first element
Test 4: Push-Pop when empty
Test 5: Normal LIFO
Test 6: Full boundary
Test 7: Push when full
Test 8: Pop from full
Test 9: Push-Pop when full
Test 10: Push-Pop with one element
Test 11: Normal Push-Pop
Test 12: Empty boundary
Test 13: Idle state
Test 14: Reset when non-empty
Test 15: Reset when full
Test 16: Random stress test
=====
Configuration: WIDTH=2 DEPTH=4
Checks performed: 1132
ALL TESTS PASSED
=====
stack_parametric_tb.v:834: $finish called at 2876000 (1ps)
```

شکل ۱۴: عبور موفق ۱۱۳۲ بررسی برای پشته پارامتری با عمق ۴ و پهنای ۲ بیت

۳.۹ نتیجه WIDTH=8, DEPTH=8

```
● mahdi@mahdi-ASUS-Zenbook-14:~/Documents/DSDL/experiment 4/quartus/stack_parametric$ vvp sim.out
VCD info: dumpfile stack_tb.vcd opened for output.
=====
STACK TEST: WIDTH=8 DEPTH=8
=====
Test 1: Reset
Test 2: Pop when empty
Test 3: Push first element
Test 4: Push-Pop when empty
Test 5: Normal LIFO
Test 6: Full boundary
Test 7: Push when full
Test 8: Pop from full
Test 9: Push-Pop when full
Test 10: Push-Pop with one element
Test 11: Normal Push-Pop
Test 12: Empty boundary
Test 13: Idle state
Test 14: Reset when non-empty
Test 15: Reset when full
Test 16: Random stress test
=====
Configuration: WIDTH=8 DEPTH=8
Checks performed: 1244
ALL TESTS PASSED
=====
stack_parametric_tb.v:834: $finish called at 3156000 (1ps)
```

شکل ۱۵: عبور موفق ۱۲۴۴ بررسی با افزایش پهنای داده به ۸ بیت

۴.۹ نتیجه WIDTH=5, DEPTH=5

```
● mahdi@mahdi-ASUS-Zenbook-14:~/Documents/DSDL/experiment 4/quartus/stack_parametric$ vvp sim.out
VCD info: dumpfile stack_tb.vcd opened for output.
=====
STACK TEST: WIDTH=5 DEPTH=5
=====
Test 1: Reset
Test 2: Pop when empty
Test 3: Push first element
Test 4: Push-Pop when empty
Test 5: Normal LIFO
Test 6: Full boundary
Test 7: Push when full
Test 8: Pop from full
Test 9: Push-Pop when full
Test 10: Push-Pop with one element
Test 11: Normal Push-Pop
Test 12: Empty boundary
Test 13: Idle state
Test 14: Reset when non-empty
Test 15: Reset when full
Test 16: Random stress test
=====
Configuration: WIDTH=5 DEPTH=5
Checks performed: 1160
ALL TESTS PASSED
=====
stack_parametric_tb.v:834: $finish called at 2946000 (1ps)
```

شکل ۱۶: عبور موفق ۱۱۶۰ بررسی برای عمق غیرتوان دو برابر ۵

نتیجه شکل ۱۶ تأیید مهمی برای فرمول پهنای اشاره گر است. در این حالت $PTR_WIDTH = \lceil \log_2(6) \rceil = 3$ می شود و اشاره گر بدون نیاز به تغییر دستی کد قادر است مقادیر صفر تا پنج را نمایش دهد.

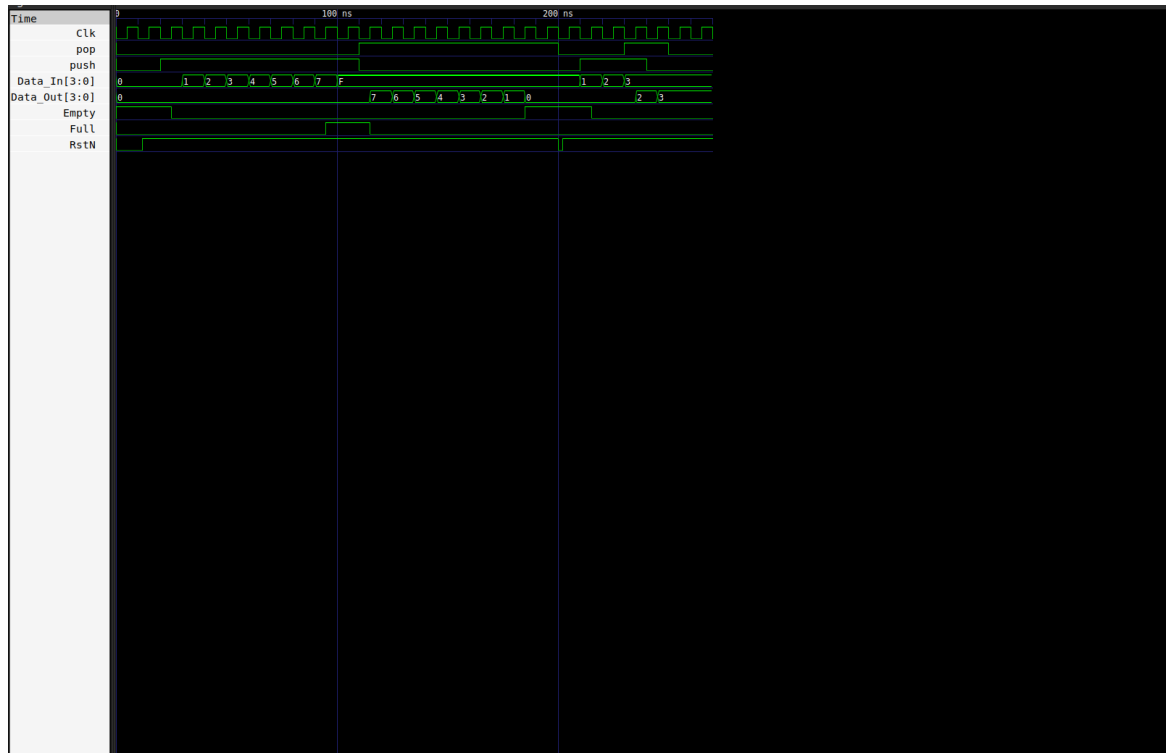
۱۰ فایل آزمون نمایشی و تحلیل شکل موج‌ها

۱.۱۰ چرا یک فایل آزمون کوتاه‌تر ساخته شد؟

فایل آزمون جامع برای ارزیابی خودکار بسیار مناسب است، اما به دلیل وجود ۱۶ حالت آزمون هدفمند و ۲۰۰ عملیات تصادفی، شکل موج کامل آن شلوغ است. برای ارائه تصویری خواناتر، فایل `stack_parametric_wave_tb.v` نوشته شد. این فایل جایگزین آزمون جامع نیست و هدف آن فقط نمایش واضح رخداد‌های اصلی روی شکل موج است. روند این فایل به ترتیب زیر است:

۱. اعمال بازنشانی و شروع از پشته خالی؛
۲. درج داده تا پر شدن کامل پشته؛
۳. یک درج اضافی در حالت پر برای نمایش حفاظت از مرز بالا؛
۴. برداشت تمام عناصر و مشاهده ترتیب LIFO تا فعال شدن Empty؛
۵. یک برداشت اضافی در حالت خالی برای نمایش حفاظت از مرز پایین؛
۶. بازنشانی دوباره؛
۷. درج دو داده و سپس فعال کردن هم‌زمان درج و برداشت؛
۸. برداشت عنصر جایگزین شده برای نمایش واضح تصمیم طراحی در حالت هم‌زمان.

۲.۱۰ شکل موج پشته پارامتری با عمق ۸ و پهنای ۴ بیت



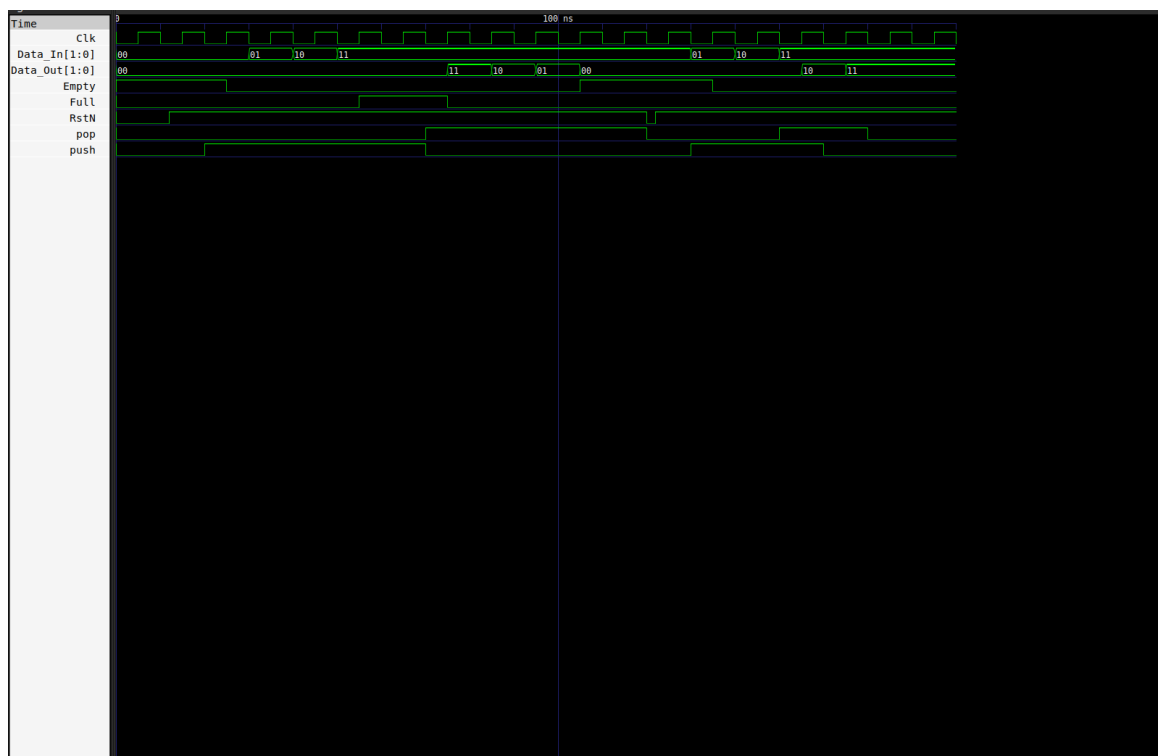
شکل ۱۷: شکل موج نمایشی پشته پارامتری با عمق ۸ و پهنای ۴ بیت

در شکل ۱۷، پس از پایان بازنشانی، درج‌های پایایی انجام می‌شوند. داده‌های صفر تا هفت وارد پشته می‌شوند و پس از درج هشتم، پرچم Full فعال می‌شود. سپس مقدار F در حالی روی ورودی قرار می‌گیرد که پشته پر است؛ چون در این وضعیت باید نادیده گرفته شود، ساختار پشته تغییر نمی‌کند.

در مرحله بعد، با فعال شدن pop، خروجی‌ها از ۷ تا صفر ظاهر می‌شوند که ترتیب معکوس درج را نشان می‌دهد. پس از برداشت آخرین عنصر، Empty فعال می‌شود. سپس یک برداشت اضافی در حالت خالی اجرا می‌شود که نباید اشاره‌گر را از صفر کمتر کند.

در بخش پایانی شکل، پس از بازنشانی مجدد، مقادیر ۱ و ۲ درج می‌شوند. سپس با Data_In=3 هر دو فرمان هم‌زمان فعال می‌شوند. در این لحظه مقدار ۲ به خروجی منتقل و رأس پشته با ۳ جایگزین می‌شود. برداشت بعدی نیز مقدار ۳ را نشان می‌دهد. بنابراین تصمیم طراحی حالت هم‌زمان به صورت مستقیم در شکل موج قابل مشاهده است.

۳.۱۰ شکل موج پشته پارامتری با عمق ۴ و پهنای ۲ بیت



شکل ۱۸: شکل موج نمایشی پشته پارامتری با عمق ۴ و پهنای ۲ بیت

شکل ۱۸ همان توالی منطقی را برای پشته کوچکتر نشان می‌دهد. این بار داده‌های دو بیتی 00، 01، 10 و 11 چهار خانه پشته را پر می‌کنند و Full پس از چهارمین درج فعال می‌شود. هنگام برداشت، خروجی‌ها به ترتیب 11، 10، 01 و 00 ظاهر می‌شوند و پس از آخرین برداشت، Empty فعال می‌شود. بخش پایانی نیز درج دو مقدار و عملیات هم‌زمان را با پهنای ۲ بیت نشان می‌دهد.

مقایسه شکل‌های ۱۷ و ۱۸ نشان می‌دهد که منطق کنترلی بدون تغییر باقی مانده، اما تعداد چرخه‌های لازم برای رسیدن به مرز پرشدن و پهنای گذرگاه داده مطابق مشخصه‌های پیمان‌ت تغییر کرده است.

۱۱ بحث و جمع‌بندی نتایج

نتایج به‌دست‌آمده سه جنبه اصلی طراحی را تأیید می‌کنند:

۱. صحت منطق پشته: فایل آزمون ثابت و فایل آزمون جامع، رفتار LIFO، مرزهای پر و خالی، درج و برداشت نامعتبر، حالت سکون، بازنشانی ناهم‌زمان و عملیات هم‌زمان را بررسی کرده‌اند.
۲. صحت پارامتری‌سازی: چهار پیکربندی متفاوت شامل عمق و پهنای اصلی، نسخه کوچکتر، پهنای بیشتر و عمق غیرتوان دو، همگی بدون تغییر الگوریتم داخلی آزمون‌ها را با موفقیت گذرانده‌اند.
۳. قابلیت سنتز: هر دو نسخه ثابت و پارامتری در Quartus با صفر خطا کامپایل شده‌اند و نماهای RTL، پس از نگاشت و پس از جایابی و تطبیق برای آن‌ها تولید شده است. کاهش واضح اندازه شبکه سخت‌افزاری در پیکربندی DEPTH=4، WIDTH=2 نیز نشان می‌دهد تغییر مشخصه‌ها واقعاً در سخت‌افزار سنتز شده منعکس شده است.

در نتیجه، طراحی نهایی خواسته اصلی آزمایش را برآورده می‌کند: یک پشته قابل سنتز که ابتدا با عمق ۸ و پهنای ۴ بیت پیاده‌سازی شده و سپس بدون تغییر ساختار داخلی به پیمانه‌ای با عمق و پهنای قابل تنظیم توسعه یافته است.

۱۲ فایل‌های تحویلی

مطابق صورت آزمایش، فایل‌های کد کامنت‌گذاری شده و فایل‌های آزمون همراه گزارش تحویل می‌شوند. فایل‌های اصلی پروژه عبارت‌اند از:

- `stack.v`: پشته ثابت با عمق ۸ و پهنای ۴ بیت؛
- `stack_tb.v`: فایل آزمون ۱۲ حالت برای نسخه ثابت؛
- `stack_parametric.v`: پشته پارامتری؛
- `stack_parametric_tb.v`: فایل آزمون جامع، خودبررسی^{۲۶} و دارای مدل مرجع؛
- `stack_parametric_wave_tb.v`: فایل آزمون کوتاه برای تولید شکل موج خوانا.

²⁶Self-Checking